CPSC-354 Report

Brandon Foley
Chapman University

December 12, 2025

Abstract

Contents

1 Introduction

2 Week by Week

2.1 Week 1

2.1.1 Homework

Here is my work on the MU Puzzle:

$$MIU \to MIUIU \to MIUIUIU \to \cdots$$

$$MI \to MII \to MIIII \to MUI \quad \Rightarrow \quad MUI$$

$$MUI \to MUIU$$

$$MUI \to MUIU \to MUIUIU \to MUIUIUIU \to MUUIU \to MUUIUIU \to \cdots$$

$$MUIIU \to MUIIUUIU \to MUIU$$

$$MUIIU \to MUUIU \to MUIU$$

$$MUIU \to MUUIU \to MUIUIU \to MUIUIUIU \to \cdots$$

$$MUIUIUIU \to MUIUIUIUIU \to MUIUIUIUIUIU \to \cdots$$

The MU Puzzle is unsolvable because it is impossible to reach the string `MU` starting from `MI` using the given rules. After a little research and thinking on my part it is the number of `I`'s that make this impossible: rule applications can double the count, add one, or remove three at a time, however, none of these operations ever reduce the number of `I`'s to exactly zero. Since `MU` has no `I`'s, it can never be derived.

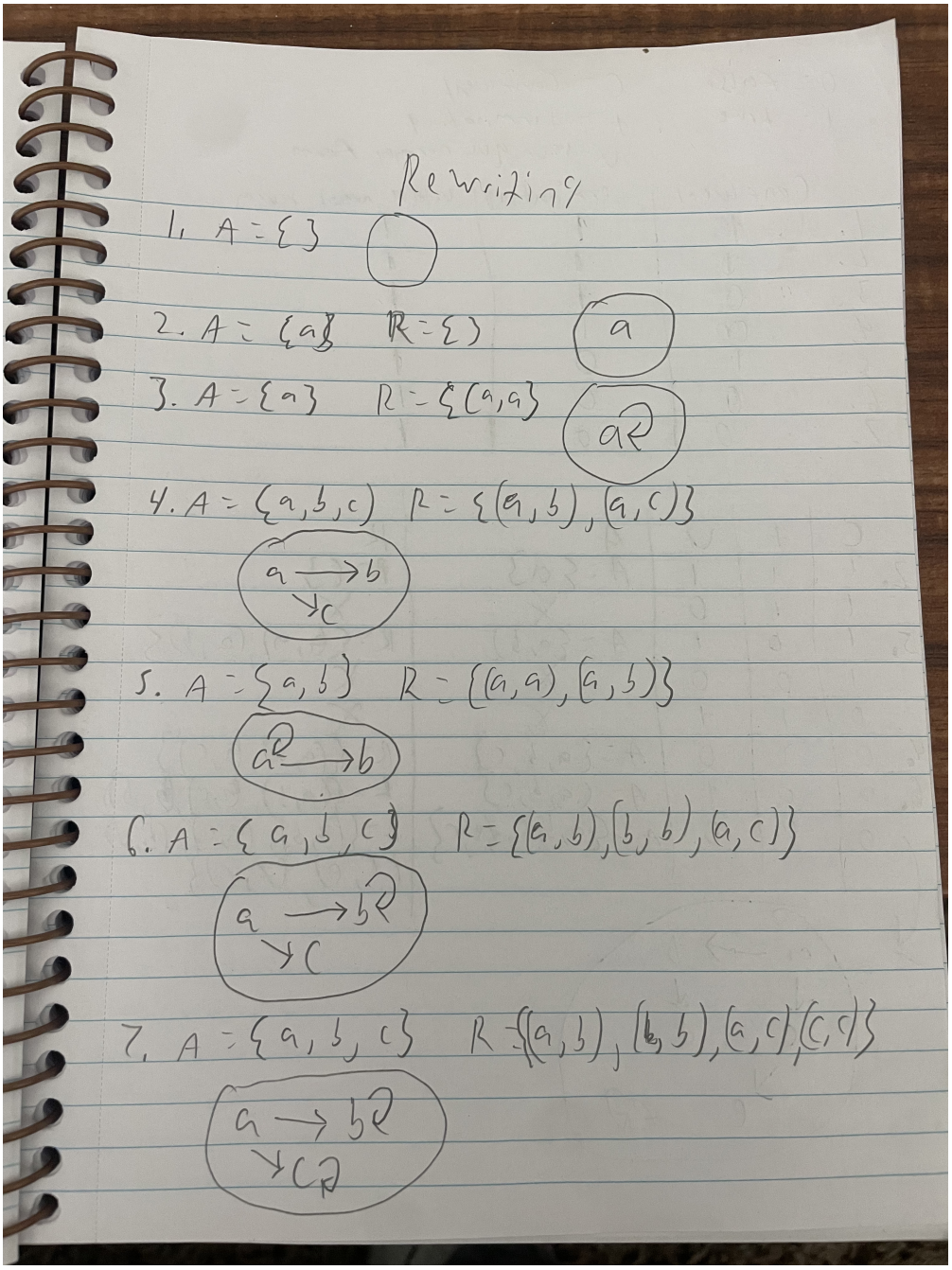
2.1.2 Questions

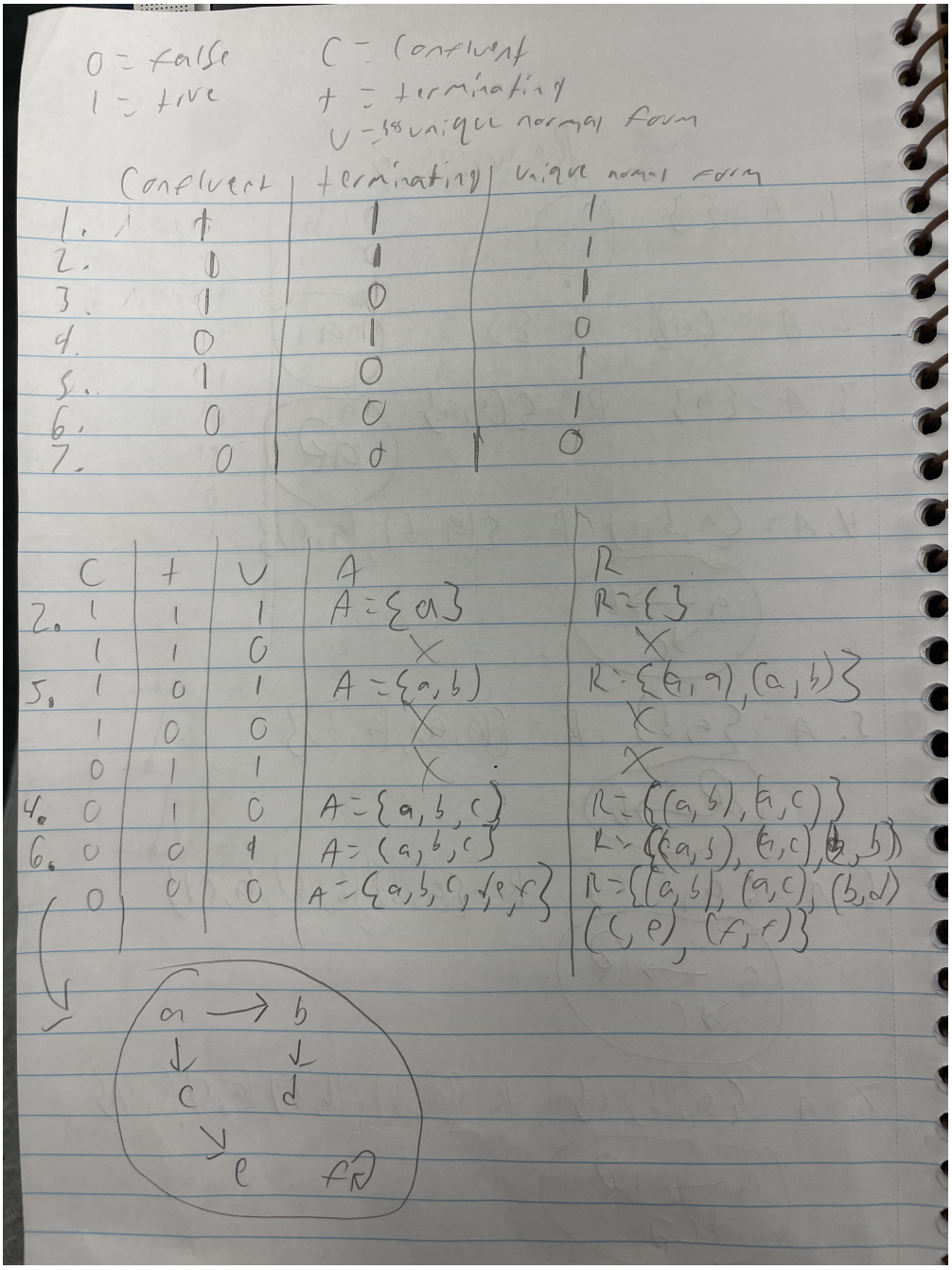
What rules could we add or remove in order to make this possible? I was thinking are there any ways to make this question solvable by adding only one more rule or maybe altering a rule. I think that this is an interesting question and concept.

2.2 Week 2

2.2.1 Homework

Here is my work on the Rewriting Homework:


Rewriting
1. A = {}
2. A = {a} R = {}
a
3. A = {a} R = {(a,a)}
aR
4. A = {a,b,c} R = {(a,b),(a,c)}
a → b
↘c
5. A = {a,b} R = {(a,a),(a,b)}
aR → b
6. A = {a,b,c} R = {(a,b),(b,b),(a,c)}
a → bR
↘c
7. A = {a,b,c} R = {(a,b),(b,b),(a,c),(c,c)}
a → bR
↘cR



In our homework we discovered 3 seperate cominations that are impossible to create. The first confluent, terminating and no unique normal form is impossible because if a system is confluent and terminating then it must have a unique normal form. The second confluent, not terminating and no unique normal form is impossible because if a system is confluent but not terminating then it must have a unique normal form when it exists. The third not confluent, terminating and unique normal form is impossible because if a system is terminating and has a unique normal form then it must be confluent.

2.2.2 Questions

When relating to programming, if a system is confluent but not terminating, like in some programs with infinite loops, how does confluence still guarantee that the result of a program is unique when it exists?

2.3 Week 3

2.3.1 Homework

Here is the work on ARS rewriting HW:

Exercise 5:

- Reduce some example strings such as `abba` and `bababa`.

$$abba \to baba \to baab \to bb \to b \to \{\}$$

$$bababa \to baabba \to bbba \to bba \to ba \to a$$

- Why is the ARS not terminating?
  The ARS is not terminating because you can infinitely go from ab to ba and vice versa.
- How many equivalence classes does $\leftrightarrow^*$ have? Can you describe them in a nice way? What are the normal forms?
  There are two equivalence classes: words with an even number of a's vs words with an odd number of a's.
  The normal forms are the empty string $\{\}$ and the string `a`.
- Can you change the rules so that the ARS becomes terminating without changing its equivalence classes?
  You can change $ba \to ab$ to $ba \to b$. This keeps the same equivalence classes but makes the system terminating.
- Write down a question or two about strings that can be answered using the ARS.
  - Does this string reduce to the empty string?
  - Are two given strings equivalent under the ARS?

Exercise 5b:

- Reduce some example strings such as `abba` and `bababa`.

$$abba \to baba \to baab \to bab \to ba \to a$$

$$bababa \to baabba \to babba \to baba \to aba \to aa \to a$$

- Why is the ARS not terminating?
  The ARS is not terminating because you can infinitely go from ab to ba and vice versa.
- How many equivalence classes does $\leftrightarrow^*$ have? Can you describe them in a nice way? What are the normal forms?
  There is one equivalence class: all words reduce to one a.
  The normal form is just `a`.

- Can you change the rules so that the ARS becomes terminating without changing its equivalence classes?
  You can change $ba \to ab$ to $ba \to b$. This keeps the same equivalence classes but makes the system terminating.
- Write down a question or two about strings that can be answered using the ARS.
  - Is there a unique normal form for this ARS?
  - Are two given strings equivalent under the ARS?

Bonus: Exercise 8

- Starting configuration:

$$\begin{aligned} &aaaaaaaaaaaaaaabbbbbbbbbbbbbbccccccccccccc \\ \to^* &\, acccccccccccccccccccccccccccccccccccccccc \\ \to &\, bbccccccccccccccccccccccccccccccccccccc \\ \to^* &\, bbbbbbbbccccccccccccccccccccccccccccccc \end{aligned}$$

- It is impossible to reach a configuration of only a's, only b's, or only c's.
- At each step, the rules correspond to the following change-vectors:

$$(a-1,\, b-1,\, c+2), \quad (a-1,\, b+2,\, c-1), \quad (a+2,\, b-1,\, c-1)$$

- Reducing these modulo 3 gives

$$(-1,\, -1,\, -1)$$

  for each move (since $+2 \equiv -1 \pmod 3$).
- Thus, we can simply subtract the numbers of a's, b's, and c's directly:

$$a - b = 15 - 14 = 1 \pmod 3$$

$$b - c = 14 - 13 = 1 \pmod 3$$

- Since these differences never reduce to 0, there will always be one leftover letter that prevents the string from reducing to a uniform string of only a's, b's, or c's.

Questions Given the rules for changing letters in Exercise 8 ARS, why is it impossible to end up with a string containing only one type of letter, and how can we use modular arithmetic to prove it?

2.4 Week 4

2.4.1 Homework

HW 4.1

Consider the following algorithm:

```
while b != 0:
    temp = b
    b = a mod b
    a = temp
return a
```

Under certain conditions this algorithm always terminates. In order for the algorithm to terminate:

- b >= 0
- a >= 0
- a and b must be integers, since doubles may cause unexpected mod results

A suitable measure function is:

$$m(a, b) = b$$

Proof of termination: Each iteration replaces b with $a \mod b$, which is always strictly smaller than the previous b but still nonnegative. Since b is a nonnegative integer, it cannot decrease indefinitely, so eventually $b = 0$ and the loop terminates.

HW 4.2

Consider the following fragment of an implementation of merge sort:

```
function merge_sort(arr, left, right):
    if left >= right:
        return
    mid = (left + right) / 2
    merge_sort(arr, left, mid)
    merge_sort(arr, mid+1, right)
    merge(arr, left, mid, right)
```

We define the measure function:

$$m(left, right) = right - left + 1$$

Proof:

- m maps to positive integers when $left \leq right$ since $m \geq 1$. If $left \geq right$ the function returns immediately (base case).
- If $left < right$ then $left \leq mid < right$, so both recursive calls shrink the measure:

  $$m(left, mid) = mid - left + 1 < right - left + 1 = m(left, right)$$

  $$m(mid + 1, right) = right - mid < right - left + 1 = m(left, right)$$

  Thus both recursive calls are on strictly smaller natural numbers.
- The natural numbers have no infinite strictly decreasing sequence, so repeated decreases of m force termination.

Therefore $m(left, right) = right - left + 1$ is a valid measure function for merge_sort: it is integer-valued, nonnegative, and strictly decreases on every recursive call.

2.4.2 Questions

Question: In a recursive backtracking algorithm, the procedure explores possible options and recurses until either a valid solution is found or all possibilities are exhausted. Since recursion can branch in many ways, how can we define a measure function that guarantees termination of the backtracking process, and what does this reveal about the relationship between the number of remaining choices and the maximum recursive depth?

2.5 Week 5

2.5.1 Homework

Here is the work on the lambda calculus evaluation:

$$(\lambda f.\, \lambda x.\, f(f(x)))\ (\lambda f.\, \lambda x.\, f(f(f\, x)))$$

Rename $f \mapsto g$ and $x \mapsto z$ in the second term:

$$(\lambda f.\, \lambda x.\, f(f(x)))\ (\lambda g.\, \lambda z.\, g(g(gz)))$$

$$\rightarrow\ (\lambda x.\, (\lambda g.\, \lambda z.\, g(g(gz)))\ ((\lambda g.\, \lambda z.\, g(g(gz)))\, x))$$

$$\rightarrow\ (\lambda x.\, (\lambda g.\, \lambda z.\, g(g(gz)))\ (\lambda z.\, x(x(xz))))$$

$$\rightarrow\ (\lambda x.\, \lambda z.\, ((\lambda z.\, x(x(xz))))(((\lambda z.\, x(x(xz))))(((\lambda z.\, x(x(xz))))z)))$$

$$\rightarrow\ (\lambda x.\, \lambda z.\, ((\lambda z.\, x(x(xz))))\ ((\lambda z.\, x(x(xz))))\ (x(x(xz)))))$$

$$\rightarrow\ (\lambda x.\, \lambda z.\, ((\lambda z.\, x(x(xz))))\ (x(x(x(x(xz))))))))$$

$$\rightarrow\ (\lambda x.\, \lambda z.\, x(x(x(x(x(x(x(xz)))))))))$$

2.5.2 Questions

Why is -equivalence considered fundamental in the lambda calculus, and how does it affect the way we define substitution and reduction?

2.6 Week 6

2.6.1 Homework

Here is the step-by-step evaluation of the factorial function in λ-calculus:

$$\text{let rec fact} = \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * \text{fact}(n-1) \text{ in fact } 3$$

$$\rightarrow (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))\, 3$$

$$\rightarrow ((\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1))\ (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1))))\ 3$$

$$\rightarrow (\lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(n-1)))\ 3$$

$$\rightarrow \text{if } 3 = 0 \text{ then } 1 \text{ else } 3 * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(3-1))$$

$$\rightarrow 3 * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1))) \; 2)$$

$$\rightarrow 3 * ((\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)) \; (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))) \; 2)$$

$$\rightarrow 3 * (\lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(n-1))) \; 2$$

$$\rightarrow 3 * (\text{if } 2 = 0 \text{ then } 1 \text{ else } 2 * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(2-1)))$$

$$\rightarrow 3 * 2 * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1))) \; 1)$$

$$\rightarrow 3 * 2 * (\lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(n-1))) \; 1$$

$$\rightarrow 3 * 2 * (\text{if } 1 = 0 \text{ then } 1 \text{ else } 1 * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(1-1)))$$

$$\rightarrow 3 * 2 * 1 * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1))) \; 0)$$

$$\rightarrow 3 * 2 * 1 * (\lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(n-1))) \; 0$$

$$\rightarrow 3 * 2 * 1 * (\text{if } 0 = 0 \text{ then } 1 \text{ else } 0 * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(0-1)))$$

$$\rightarrow 3 * 2 * 1 * 1 = 6$$

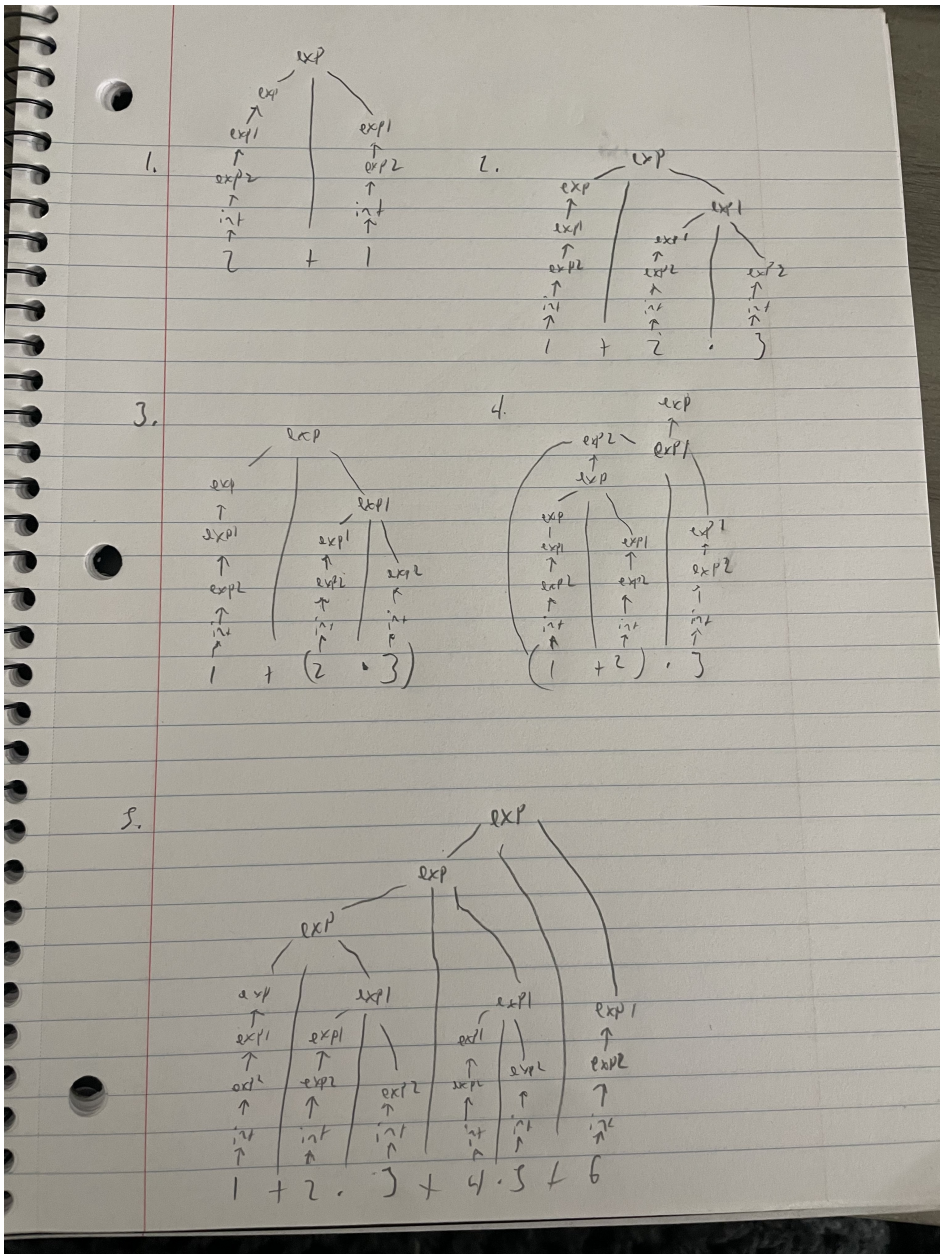
2.6.2 Questions

In this example, the fix operator allows us to define recursive behavior without explicit recursion syntax. How does the use of fix in λ-calculus compare to the use of recursion in functional programming languages, and why is this fixed-point construction needed for expressing general recursive functions in a purely functional system?

2.7 Week 7

2.7.1 Homework

Here is my homework for context free gramar trees:



2.7.2 Questions

In context-free grammars, how does introducing precedence levels (like Exp, Exp1, Exp2) formally enforce operator precedence compared to simply defining Exp -> Exp '+' Exp | Exp '*' Exp | Integer?

2.8 Week 8

2.8.1 Homework

Here is my work for the Lean proofs from the report.txt file:

Level 5:

```
rw [add_zero]
rw [add_zero]
rfl
```

Level 6:

```
rw [add_zero c]
rw [add_zero b]
rfl
```

Level 7:

```
rw [one_eq_succ_zero]
rw [add_succ]
rw [add_zero]
rfl
```

Level 8:

```
nth_rewrite 2 [two_eq_succ_one]
rw [add_succ]
rw [succ_eq_add_one]
nth_rewrite 1 [two_eq_succ_one]
rw [succ_eq_add_one]
rw [four_eq_succ_three]
nth_rewrite 1 [succ_eq_add_one]
rw [three_eq_succ_two]
rw [succ_eq_add_one]
rw [two_eq_succ_one]
rw [succ_eq_add_one]
rfl
```

2.8.2 Natural Language Proof (for Level 7)

In natural lanuguage we start with the statement that $1 + a = \text{succ}(a)$, where $\text{succ}(x)$ denotes the successor of x (that is, $x + 1$).

1. By definition, $1 = \text{succ}(0)$.
2. Substitute this into the left-hand side: $\text{succ}(0) + a$.
3. Using the recursive definition of addition, $\text{succ}(m) + n = \text{succ}(m + n)$. So, $\text{succ}(0) + a = \text{succ}(0 + a)$.
4. Next, by the base case of addition, $0 + a = a$.
5. Therefore, $\text{succ}(0 + a) = \text{succ}(a)$.
6. Hence, $1 + a = \text{succ}(a)$.

this shows that the proof follows directly from the base and inductive definitions of addition and the successor function in arithmetic.

2.8.3 Questions

When mathematicians informally rely on "obvious" or "intuitive" steps (like symmetry or associativity), how can such steps be represented in formal verification systems?

2.9 Week 9

2.9.1 Homework

Here is my work for the Lean induction proofs:

Exercise 1:

```
induction n
rw [add_zero]
rfl
rw [add_succ]
rw [n_ih]
rfl
```

Exercise 2:

```
induction b
repeat rw [add_zero]
rfl
repeat rw [add_succ]
rw [n_ih]
rfl
```

Exercise 3:

```
induction b
rw [add_zero]
rw [zero_add]
rfl
rw [add_succ]
rw [succ_add]
rw [n_ih]
rfl
```

Exercise 4:

```
induction c
repeat rw
[add_zero]
rfl repeat
rw [add_succ]
rw [n_ih]
rfl
```

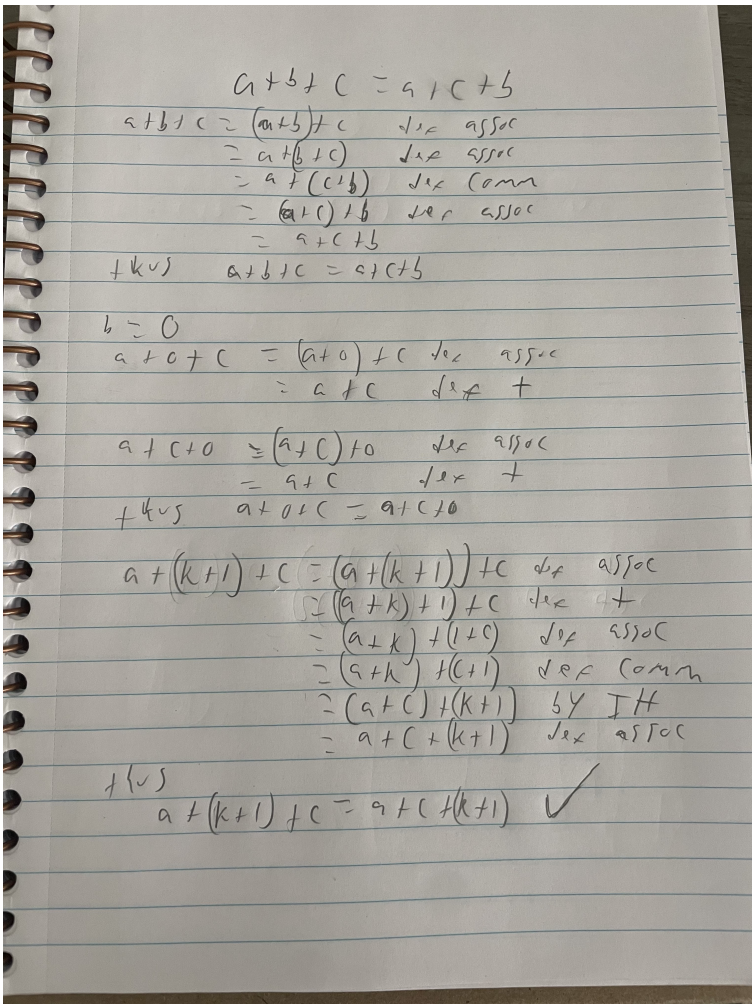
Exercise 5 (No Induction):

```
rw [add_assoc]
rw [add_assoc]
rw [add_comm b c]
rfl
```

Exercise 5 (Induction):

```
induction b
rw [add_zero]
rw [add_zero]
rfl rw [add_succ]
rw [succ_add]
rw [add_succ]
rw [n_ih]
rfl
```

Pen and paper proof for exercie 5:



2.9.2 Questions

When performing induction in Lean, why is it necessary to explicitly handle both the base and inductive cases, and how does Lean's treatment of recursive definitions ensure that each step of the induction corresponds to a structurally smaller argument?

2.10 Week 10

This week, we continued working in Lean, focusing on constructing proofs using `exact` statements and lambda expressions (λ).

1. **Exercise 6:**
$$\texttt{exact } \lambda c \mapsto \texttt{and\_intro } c \gg h$$
2. **Exercise 7:**
$$\texttt{exact } \lambda \langle c, d \rangle \mapsto h\ c\ d$$
3. **Exercise 8:**
$$\texttt{exact } \lambda (s : S) \mapsto \texttt{and\_intro } (h.\text{left } s)\ (h.\text{right } s)$$
4. **Exercise 9:**
$$\texttt{exact } \lambda (r : R) \mapsto \texttt{and\_intro } (\lambda (\_ : S) \mapsto r)\ (\lambda (\_ : \neg S) \mapsto r)$$

2.10.1 Questions

Question: How does Lean use λ-abstraction and `and_intro` to represent logical conjunction ($\wedge$) in proofs?

2.11 Week 11

This week, we continued working in Lean, focusing on constructing proofs using negation.

1. **Exercise 9:**
$$\texttt{exact } \lambda (a : P \wedge A) \mapsto h\ a.\text{left}\ a.\text{right}$$
2. **Exercise 10:**
$$\texttt{exact } \lambda (p : P)\ (a : A) \mapsto h\ (\texttt{and\_intro } p\ a)$$
3. **Exercise 11:**
$$\texttt{exact } \lambda a \mapsto h\ (\lambda n a \mapsto n a\ a)$$
4. **Exercise 12:**
$$\texttt{exact } \lambda n b \mapsto h\ (n b \gg \texttt{false\_elim})$$

2.11.1 Questions

Question: How can falsification be modeled within Lean's constructive logic, where the law of excluded middle does not hold, and what does this reveal about representing scientific reasoning in type theory?

2.12 Week 12

2.12.1 Homework

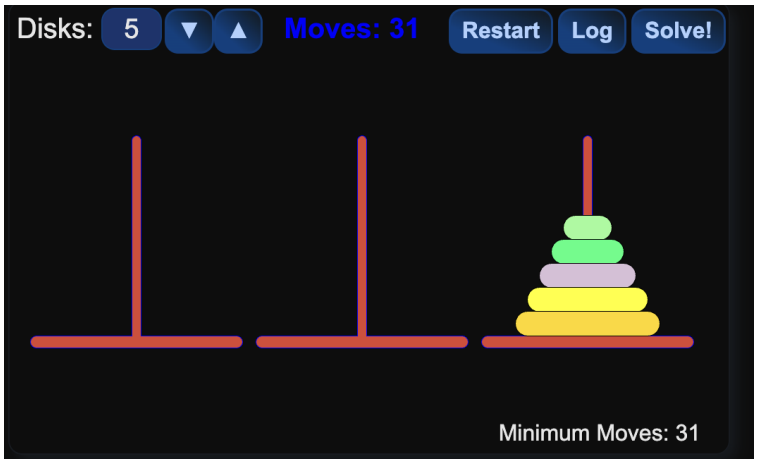
Here is my work on the Towers of Hanoi problem with 5 disks:

```
hanoi 5 0 2
    hanoi 4 0 1
        hanoi 3 0 2
            hanoi 2 0 1
                hanoi 1 0 2 = move 0 2
                move 0 1
                hanoi 1 2 1 = move 2 1
            move 0 2
            hanoi 2 1 2
                hanoi 1 1 0 = move 1 0
                move 1 2
                hanoi 1 0 2 = move 0 2
        move 0 1
        hanoi 3 2 1
            hanoi 2 2 0
                hanoi 1 2 1 = move 2 1
                move 2 0
                hanoi 1 1 0 = move 1 0
            move 2 1
            hanoi 2 0 1
                hanoi 1 0 2 = move 0 2
                move 0 1
                hanoi 1 2 1 = move 2 1
    move 0 2
    hanoi 4 1 2
        hanoi 3 1 0
            hanoi 2 1 2
                hanoi 1 1 0 = move 1 0
                move 1 2
                hanoi 1 0 2 = move 0 2
            move 1 0
            hanoi 2 2 0
                hanoi 1 2 1 = move 2 1
                move 2 0
                hanoi 1 1 0 = move 1 0
        move 1 2
        hanoi 3 0 2
            hanoi 2 0 1
                hanoi 1 0 2 = move 0 2
                move 0 1
                hanoi 1 2 1 = move 2 1
            move 0 2
            hanoi 2 1 2
                hanoi 1 1 0 = move 1 0
                move 1 2
                hanoi 1 0 2 = move 0 2
```

```
move 0 2
move 0 1
move 2 1
move 0 2
move 1 0
move 1 2
move 0 2
move 0 1
move 2 1
move 2 0
move 1 0
move 2 1
move 0 2
move 0 1
move 2 1
move 0 2
move 1 0
move 1 2
move 0 2
move 1 0
move 2 1
move 2 0
move 1 0
move 1 2
move 0 2
move 0 1
move 2 1
move 0 2
move 1 0
move 1 2
move 0 2
```

The Towers of Hanoi solution for 5 disks requires 31 moves, which follows the formula $2^n - 1$ where n is the number of disks.

Here is a visualization of the completed Towers of Hanoi with all 5 rings moved from peg 1 to peg 3:



2.12.2 Questions

How does the recursive solution for n disks relate to the mathematical proof that the minimal number of moves required is $2^n - 1$, and what insights does this provide about the relationship between recursive algorithms and mathematical induction?

2.13 Week 13

2.13.1 Homework

Question 1: Does our Python implementation of lambda calculus conforms to its specification given by its mathematical theory?

After reviewing the specifications of lambda calculus and comparing it to our Python implementation, I think that there are some discrepancies that indicate our implementation does not fully conform to the specification.

Question 2: Testing Lambda Expressions

I created some more test cases in `test.lc`:

```
// Basic identity function
(\x.x) a

// Function application associativity
a b c d
(a)

// Church booleans
(\x.\y.x) a b // true - should return a
(\x.\y.y) a b // true - should return b

// Church numerals and arithmetic
(\f.\x.f(f x)) // Church numeral 2
(\f.\x.f(f(f x))) // Church numeral 3

// Multiplication: 2 * 3 = 6
((\m.\n. m n) (\f.\x. f (f x))) (\f.\x. f (f (f x)))
```

Expected vs Actual Results:

- `(\x.x) a` → `a` (identity function applied to a)
- `a b c d` → `(((a b) c) d)` (left-associativity)
- `(a)` → `a` (parentheses don't change meaning)
- `(\x.\y.x) a b` → `a` (true returns first argument)
- `(\x.\y.y) a b` → `b` (true returns second argument)
- Church numeral stopped after certain amount of reductions due to implementation limits (should represent 2 and 3)

Question 3: Capture Avoiding Substitution

The substitution algorithm works as follows:

```
def substitute(tree, name, replacement):
```

```
if tree[0] == 'var':
if tree[1] == name:
return replacement # n [r/n]  r
else:
return tree # x [r/n]  x
elif tree[0] == 'lam':
if tree[1] == name:
return tree # \n.e [r/n]  \n.e
else:
fresh_name = name_generator.generate()
return ('lam', fresh_name,
substitute(substitute(tree[2], tree[1], ('var', fresh_name)),
name, replacement))
# \x.e [r/n]   (\fresh.(e[fresh/x])) [r/n]
elif tree[0] == 'app':
return ('app', substitute(tree[1], name, replacement),
substitute(tree[2], name, replacement))
```

Key Insight: When substituting into `\x.e` where `x  name`, we generate a fresh variable to avoid capture, rename `x` to the fresh name in the body, then perform the substitution.

Question 5: Minimal Working Example that doesn't reduce to normal form

The smallest meaningful MWE that fails in the original interpreter:

```
a ((\x.x) b)
```

Why this fails:

- Current behavior: `a ((\x.x) b)` stays as `a ((\x.x) b)`
- Expected behavior: Should reduce to `a b`
- Reason: The original interpreter only evaluates the function position, not the argument position

Question 7: Tracing Evaluation Steps

Tracing `((\m.\n.  m n) (\f.\x.  f (f x))) (\f.\x.  f x)`:

```
((\m.\n. m n) (\f.\x. f (f x))) (\f.\x. f x)
→ ((\n. (\f.\x. f (f x)) n)) (\f.\x. f x)
→ ((\f.\x. f (f x)) (\f.\x. f x))
→ (\x. (\f.\x. f x) ((\f.\x. f x) x))
→ (\x. (\f.\x. f x) (\x. x x)) [incorrect - demonstrates capture issue]
```

The trace revealed that the original interpreter didn't fully reduce expressions due to incomplete evaluation strategy.

Quetion 8: Recursive Evauluation Trace

```
12: eval(((\m.\n. m n) (\f.\x. f (f x))) (\f.\x. f x))
    12: eval((\m.\n. m n) (\f.\x. f (f x)))
        12: eval(\m.\n. m n)
            return: (\m.\n. m n)
        12: eval(\f.\x. f (f x))
            return: (\f.\x. f (f x))
        app of lam: substitute((\n. m n), m, (\f.\x. f (f x)))
        39: substitute((\n. m n), m, (\f.\x. f (f x)))
```

```
            39: substitute(\n. m n, m, (\f.\x. f (f x)))
                39: substitute(m n, m, (\f.\x. f (f x)))
                    39: substitute(m, m, (\f.\x. f (f x)))
                        return: (\f.\x. f (f x))
                    39: substitute(n, m, (\f.\x. f (f x)))
                        return: n
                    return: ((\f.\x. f (f x)) n)
                return: (\n. ((\f.\x. f (f x)) n))
        return: (\n. ((\f.\x. f (f x)) n))
    12: eval((\n. ((\f.\x. f (f x)) n)) (\f.\x. f x))
        app of lam: substitute(((\f.\x. f (f x)) n), n, (\f.\x. f x))
        39: substitute(((\f.\x. f (f x)) n), n, (\f.\x. f x))
            39: substitute((\f.\x. f (f x)), n, (\f.\x. f x))
                return: (\f.\x. f (f x))
            39: substitute(n, n, (\f.\x. f x))
                return: (\f.\x. f x)
            return: ((\f.\x. f (f x)) (\f.\x. f x))
        return: ((\f.\x. f (f x)) (\f.\x. f x))
    12: eval((\f.\x. f (f x)) (\f.\x. f x))
        12: eval(\f.\x. f (f x))
            return: (\f.\x. f (f x))
        12: eval(\f.\x. f x)
            return: (\f.\x. f x)
        app of lam: substitute(\x. f (f x), f, (\f.\x. f x))
        39: substitute(\x. f (f x), f, (\f.\x. f x))
            39: substitute(f (f x), f, (\f.\x. f x))
                39: substitute(f, f, (\f.\x. f x))
                    return: (\f.\x. f x)
                39: substitute((f x), f, (\f.\x. f x))
                    39: substitute(f, f, (\f.\x. f x))
                        return: (\f.\x. f x)
                    39: substitute(x, f, (\f.\x. f x))
                        return: x
                    return: ((\f.\x. f x) x)
                return: ((\f.\x. f x) ((\f.\x. f x) x))
            return: (\x. ((\f.\x. f x) ((\f.\x. f x) x)))
        return: (\x. ((\f.\x. f x) ((\f.\x. f x) x)))
    12: eval(\x. ((\f.\x. f x) ((\f.\x. f x) x)))
        return: (\x. ((\f.\x. f x) ((\f.\x. f x) x)))
final result: (\x. ((\f.\x. f x) ((\f.\x. f x) x)))
```

In the trace above i substituted the actual letter vs what the computer using var1-5 to represent variables internally.

2.13.2 Questions

In lambda calculus, the choice between applicative order (evaluating arguments first) and normal order (evaluating function first) evaluation strategies can lead to different termination behavior. How does this choice in our interpreter implementation reflect the fundamental trade-offs between efficiency and completeness in computational systems, and what does this reveal about the relationship between evaluation strategies and the Church-Rosser theorem in guaranteeing consistent results regardless of evaluation order?

3 Essay

When I started this course, I thought of programming languages mainly as tools for getting computers to do things, run algorithms, and a little bit of how tools and lanugages like python work. Lambda calculus was interesting in a theoretical way, but it felt very different from the code I write every day. With lean, the abstract concepts about proofs, computation, and logic came together in a way that actually made sense for real programming.

The first thing that I truly understood was how Lean forced me to be precise about things I normally take for granted. In Week 8, when we had to prove basic arithmetic facts like 1 + a = succ(a), I realized I'd been using these properties for years without ever thinking about why they're true. Writing the Lean proof: theorem $one_add_eq_succ : (a :), 1+a = Nat.succa := byintroarw[show(1 :) = Nat.succ0byrfl]rw[add_succ]rw[add_zero]mademeactuallyt$

This reminded me of when we implemented the lambda calculus interpreter in Python. There were moments where the interpreter wouldn't reduce an expression that "obviously" should reduce, and I had to debug by tracing through exactly how substitution worked.

The interesting part to me came in Weeks 10 and 11 when we started proving logical statements. The fact that P → Q (P implies Q) is represented as a function from proofs of P to proofs of Q was super interesting to me.

When I wrote:

theorem $and_comm : PQßQP := byintrohexact<h.right, h.left>$

I realized this wasn't just proving something about logic; it was writing a program that transforms evidence. The angle brackets ⟨h.right, h.left⟩ aren't just notation, they're actually constructing a proof of Q P from the components of the proof of P Q, like constructing a tuple in Python or another programing language.

This connected back to our work with lambda calculus, where functions aren't just computation devices but can represent logical reasoning too. It made me think differently about the code I write: every time I create a function that transforms data, there's a logical interpretation of what that function guarantees about the relationship between input and output. Theres also this level of abstarction that I never really thought about before.

At first, Lean felt like just another academic exercise. But the seminar I attended with Shaun Andrews from Boeing changed my perspective. He talked about data validation in aircraft systems and how they need to be absolutely certain about their analysis. While he wasn't using Lean specifically, the same need for justification applies and it made me think about how lean could be used in real-world scenarios.

In my own internship, I've worked with dashboards that track system performance. A small bug in how data gets aggregated could lead to wrong decisions being made. Working with Lean made me think, what if we could prove that our data transformations preserve certain properties? What if we could guarantee that our aggregations are commutative and associative (like we proved addition to be)?

The experience also changed how I debug. Now when I hit a weird bug, I find myself thinking: "What are my assumptions here? What exactly does this function promise about its output?" It's like doing mental Lean proofs—breaking down why I think something should work, and looking for where my reasoning might have a hole. We also learned how to trace inside of the debugger and that has helped me a lot as well.

I used to think formal verification was only for aerospace or medical devices, super high-stakes systems where failure isn't an option. Now I see it differently. Even in everyday web development, being able to reason precisely about your code matters. The concepts we learned in Lean—dependent types, constructive logic, inductive proofs, are making their way into practical tools.

Languages like TypeScript are getting more sophisticated type systems that can catch more errors at compile time. Rust's ownership system feels like it's enforcing logical properties about resource usage. These aren't

theorem provers, but they're inspired by the same ideas about using the type system to prove things about your program.

Lean started as just another tool to learn in this course, but it ended up changing how I think about programming fundamentally. It helped me bridge the gap between the abstract theory we learned in the first half of the semester and the practical programming I do outside class. The biggest takeaway wasn't how to use Lean specifically, but learning to think more carefully about assumptions, proofs, and guarantees in my code.

Software keeps getting more complex and more critical to everyday life, this kind of rigorous thinking has become extremly important to the workforce. Whether I'm working on a class project, an internship, or eventually a job, this mindset of breaking things down to first principles and proving they work, not just hoping they do.

4 Evidence of Participation

- Attended all lectures and actively participated in discussions.
- Completed weekly homework assignments and submitted them on time.

Seminar attendance:

In a seminar I attended I heard from Shaun Andrews, a Senior Data Analyst at Boeing and a Chapman University alumnus. Shaun shared valuable insights into his career path, explaining how the role of a data analyst can evolve into data science and beyond. He walked us through the full data process, from data cleaning and preparation to analysis, visualization, and communicating findings to stakeholders. He emphasized that advancing in the field is not only about technical skills, but also about learning how to tell meaningful stories with data. A major focus of his talk was data visualization. He highlighted how important it is to transform complex datasets into clear, impactful visuals that drive decision-making. His primary tool that he showed us is Tableau, and he demonstrated how he uses dashboards to monitor trends and support large-scale projects at Boeing. He also walked us through a case study involving aircraft sensor data, showing how visualization helped identify unusual patterns that improved predictive maintenance. I found a strong connection to my own internship experience, where I used Grafana and Power BI to build performance dashboards and monitor key metrics. Shaun briefly discussed Power BI as well, showing how these visualization tools are used across industries.

5 Conclusion

Looking back, this course connected a lot of dots for me between theory and the code I actually write. Starting with the puzzles like MU and abstract rewriting systems it was super abstract at first, but it framed a really important concept. How do we know a set of rules (or a program) will eventually stop and give us a answer? Tracing lambda calculus reductions by hand, and then seeing the same concepts break down in our Python interpreter project at the end, showed us why formal models matter.

The shift to Lean was a also a very interesting part of what we learned. Having the system reject my "obvious" steps forced me to understand the exact logic behind things like induction or commutativity, which I normally take for granted. It made me appreciate the precision required for verification in critical systems, like the ones discussed in the Boeing seminar I attended.

The most useful takeaway is a new lens for problem-solving. I now think more about the underlying "rewrite system", the state changes, termination conditions, and equivalence classes of my data and how code are run. This mindset is directly applicable to software engineering, whether I'm designing an API, optimizing an algorithm, or debugging a complex state machine. For future iterations of the course, I'd suggest even earlier hands-on projects and programming assignments, to help understand the theory immediately.

References

[1] Microsoft Research. "The Lean Theorem Prover." Lean Community, 2023. https://leanprover.github.io/

[BLA] Author, Title, Publisher, Year.